Lab 2: Interpreting Statements

Goal: Extend your expression interpreter from Lab 1 to support a complete MiniJava program: variable declarations, scoping, assignment operators, and control flow statements (`if`, `while`, `for`, `break`, `continue`).

Deadline: 12:00:00 Apr 27, 2026 (Hard Deadline, No extension)

Programming Language: Java Only.

Task 1: Port Your Lab 1 Implementation

1.1 What Changed in the Grammar

In Lab 1, the entry point was a single expression:

```
compilationUnit : expression EOF
```

In Lab 2, the entry point is a **block** — a sequence of statements enclosed in braces:

```
compilationUnit : block EOF
```

The grammar has been extended with new rules for blocks, statements, and variable declarations.

The project structure is identical to Lab 1:

```
lab2-statments/
├── pom.xml
├── src/main/java/cn/edu/nju/cs/
│   ├── Main.java                      # Entry point
│   ├── MiniJavaLexer.g4               # Updated lexer grammar
│   ├── MiniJavaParser.g4              # Updated parser grammar
│   ├── MiniJavaLexer.java             # Generated (do not modify)
│   ├── MiniJavaParser.java            # Generated (do not modify)
│   ├── MiniJavaParserBaseVisitor.java # Generated base visitor to extend
│   └── ...
└── testcase/
    ├── ForLoop-1.mj
    ├── IfStatement-1.mj
    └── ...
```

1.2 What You Need to Do

- You can cpoy the `testcase` and `MiniJava*.g4` to your existing project of Lab 1, do not forget to regenerate the ANTLR sources after updating the grammar files.
- Adapt your expression visitor methods to the new grammar if needed (some production rules may have changed slightly).

Task 2: Understand the New Grammar

2.1 Program Structure

A MiniJava program is a single top-level block:

```
compilationUnit : block EOF

block : '{' blockStatement* '}'

blockStatement
    : localVariableDeclaration ';'
    | statement
    ;
```

A `block` contains zero or more `blockStatement`s. Each `blockStatement` is either a local variable declaration or a statement.

2.2 Variable Declarations

```
localVariableDeclaration
    : primitiveType identifier '=' expression   // initialized
    | primitiveType identifier                  // uninitialized
    ;
```

- **Initialized** — the `expression` is evaluated and stored to `identifier` immediately.
- **Uninitialized** — the variable exists in the current scope with the default value for its type.

Primitive Type	Default Value
`int`	0
`char`	(char)0
`boolean`	false
`string`	""

```
{
    int x = 1 + 2;   // x = 3
    boolean flag;     // flag (uninitialized)
    flag = true;      // now flag = true
}
```

Note 1: Declaring the same identifier twice **in the same scope** will lead an Error.

2.3 Identifiers in Expressions

In Lab 2, an identifier appearing as a primary expression resolves to the variable's current value:

```
primary : '(' expression ')' | literal | identifier
```

Note 1: Accessing an undeclared identifier will lead to an Error.

Task 3: Implementing the Interpreter

Note: Do not throw any class which implements `Error` at any time. Use `Exception` instead. Throwing an `Error` will lead to a Wrong Answer in our automatic testing.

3.1 Scoping

Each `block` creates a new scope on entry. When a block exits, print each variable declared in **that block's scope** in **alphabetical order**:

Note: An inner-scope variable with the same name as an outer one is allowed. Only the innermost match can be accessed in that scope.

```
{
    int x = 375;
    {
        int x = 321;   // shadows outer x
    }
}
```

3.2 Assignment Operators

Lab 2 introduces assignment as an expression. The following assignment operators are supported:

```
=   +=   -=   *=   /=   %=   &=   |=   ^=   <<=   >>=   >>>=
```

Types must be compatible. `int` and `char` are mutually compatible; all others must match exactly.

LHS type	RHS type	Behavior
`int`	`int`	Store as `int`
`int`	`char`	Promote `char` → `int`, store
`char`	`int`	Promote `char` → `int` for calculation, store as `char`
`char`	`char`	Store as `char`
`boolean`	`boolean`	Store as `boolean`
`string`	`string`	Store as `string`
other	any	Error

Note 1: The LHS must be a **identifier**.

Note 2: Only assign to a variable is allowed; otherwise, it will lead to an Error.

Note 3: Both operands must be integral (`int` or `char`). The operation is performed as `int`, then the result is cast back to the declared type of target .

Note 4: For string, only `=` and `+=` are supported, convert the non-string operand to a string if necessary.

```
string s = "foo";
s += 42;          // s = "foo42"
char c = 'z';
c += 1;           // c = '{' (ASCII 123)


int a = 309;
int b = -772;
a += b;    // a = -463
```

3.3 `if` / `else`

Note 1: The condition must be of type `boolean`;

Note 2: Only the selected branch is executed;

Note 3: `else if` is not a special keyword — it is `else` followed by an `if` statement;

```
{
    int n = 5;
    if (n < 0) {
        int r = 0;
    } else if (n == 0) {
        int r = 1;
    } else {
        int r = 2;
    }
}
```

Output:

```
Scope 1: r: (int) 2
Scope 0: n: (int) 5
```

3.4 `while`

Note 1: The condition must be of type `boolean`;

Note 2: If the body is a block, it creates a **new scope** each iteration, printed on exit at the end of that iteration (even if `break` or `continue` is hit).

```
{
    int i = 0;
    while (i < 2) {
        int x = i;
        i = i + 1;
    }
}
```

Output:

```
Scope 1: x: (int) 0
Scope 1: x: (int) 1
Scope 0: i: (int) 2
```

3.5 `for`

Note 1: If `forInit` declares a variable, that variable lives in the **for loop's own implicit scope**

Note 2: The body block (if present) creates its **own scope per iteration**

Note 3: If the body is just an expression statement rather than a block, there's no need to open a new scope for it. Since the expression never introduces new variables.

```
{
    for (int i = 0; i < 2; i++) {
        int x = i;
    }
}
```

Output:

```
Scope 2: x: (int) 0
Scope 2: x: (int) 1
Scope 1: i: (int) 2
```

3.6 `break` and `continue`

When `break` or `continue` is executed, every block scope between the statement and the loop boundary is exited in order

Note: Both statements are only valid **inside a loop** (`while` or `for`). Using them outside any loop will lead to an Error.

```
{
    while (true) {
        int x = 5;
        {
            int y = 10;
            break;
        }
    }
}
```

Output:

```
Scope 2: y: (int) 10
Scope 1: x: (int) 5
```

Task 4: Input and Output Format

Your interpreter takes a `.mj` file as input.

Error output — if any runtime error occurs, the interpreter prints **Error message** `Process exits with 34.` **to** `System.out` and exits with code `34`.

Note : Please print the Error message `Process exits with 34.` and the normal output to `System.out`. This is important for our automatic testing. Besides, you can use `System.err` to print additional debug information if needed, it will not be considered in the correctness of the output.

Output Format for Scopes

When a block exits, print each variable declared in **that block's scope** in **alphabetical order**:

```
Scope <depth>: <name>: (<type>) <value>
```

- `<depth>` is the nesting depth of the block, starting at `0` for the outermost block.
- Only variables from the **current** scope are printed;

Type	Value display	Example line
`int`	Decimal integer	`Scope 0: a: (int) 42`
`char`	Bare character (no quotes)	`Scope 0: c: (char) A`
`boolean`	`true` / `false`	`Scope 0: b: (boolean) true`
`string`	Bare string (no quotes)	`Scope 0: s: (string) hello`

```
Scope 1: y: (int) 110
Scope 1: z: (int) 616
Scope 0: x: (int) 506
```

Task 5: Submission

Submit your source files as a zip to our OJ system. OJ will use the maven files to build the interpreter. OJ then tests the correctness of the output. We do not consider the efficiency of the interpreter in this lab.

The `.zip` should follow the same structure as Lab 1:

```
submission.zip
└── outer_folder/
    ├── src/
    └── pom.xml
```